

DFS - min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty() do
 x = q.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 g.enqueue(y)
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

Path creation
 (prev, start, end)
 if end not in prev and
 s != t then "no path"
 list path = []
 while current != end
 path.add(current)
 current = prev[current]
 path.add(start)
 path.reverse
 return path
 O(V)

DFS - max len path
 longest time for activity
 (G) list order = top-sort(G)
 dict dist, prev
 for v in G.vertices
 dist[v] = -infinity
 dist[start] = 0
 for u in order
 if dist[u] != inf
 for v in Nout(u) do
 if dist[u] + cost(u,v) > dist[v]
 dist[v] = dist[u] + cost(u,v)
 prev[v] = u
 return dist, prev
 O(V+E)

DAG - Topo Sort
 order tasks in s.t. u -> v
 u comes before v (G)
 list sorted_order = []
 queue q: dict count
 for each v in G.vertices
 count[v] = indegree(G, v)
 if count[v] == 0: g.enqueue(v)
 while not q.empty():
 x = g.dequeue()
 sorted_order.append(x)
 for y in Nout(x)
 count[y] --
 if count[y] == 0:
 g.enqueue(y)
 return sorted_order
 O(V+E)

Kosaraju - SCC (G) -> SCC
 - funct DFS:
 for y in Nout(x)
 if y not in visited:
 visited.add(y)
 DFS(y, visited, precomp)
 precomp.append(x)
 - alg (precomp, comp)
 for s in range(nodiv)
 if s not in visited:
 visited.add(s)
 DFS(s, visited, precomp)
 visited.clear() c = 0
 while precomp:
 A = precomp.pop()
 if A not in visited:
 c += 1
 g.enqueue(A)
 return comp
 O(V+E)

Min len cycle (G)
 min_cycle = inf
 for start in vertices // BFS for every
 dict dist queue g: enqueue(start)
 dist[start] = 0
 while not g.empty():
 u = g.dequeue()
 for v in Nout(u)
 if v == start min_cycle = min(min_cycle, dist[u] + 1)
 else if v not in dist:
 dist[v] = dist[u] + 1
 g.enqueue(v)
 return min_cycle
 O(V * (V+E))

Dijkstra -> from s to all nodes
 (G, s)
 priority queue pq: dist, dist_prev
 for each vertex v in G: vertices
 dist[v] = infinity
 dist[s] = 0
 pq.insert(s, 0)
 while not pq.empty():
 u = pq.extract_min() // O(E log V)
 for v in Nout(u) do:
 if dist[u] + cost(u,v) < dist[v]
 dist[v] = dist[u] + cost(u,v)
 prev[v] = u
 pq.insert(v, dist[v])
 return dist, prev
 O(V log V)

Floyd - Warshall all-all
 - shortest paths (G)
 matrix dist: dist[i][j] = cost(i,j)
 for v in G.vertices: dist[v][v] = 0
 for (u,v) in G.edges
 dist[u][v] = cost(u,v)
 for k in G.vertices
 for i in G.vertices
 for j in G.vertices
 if dist[i][k] + dist[k][j] < dist[i][j]
 dist[i][j] = dist[i][k] + dist[k][j]
 return dist
 O(V^3)

DFS - Full version -> max flow
 (G, t, capacity)
 Residual Graph GR
 max_flow = 0
 while paths from s to t
 bottleneck = min cap. from paths
 for each edge (u,v) in paths
 GR.capacity[u][v] -= bottleneck
 GR.capacity[v][u] += bottleneck
 max_flow += bottleneck
 return max_flow
 O(E * min(V, E))

Tarjan -> for SCC (G)
 int index = 0
 stack st
 dict dfs_num, dfs_low
 set onstack list all_sccs = []
 for v in G.vertices
 if v not in dfs_num
 Tarjan-Visited(G, v, idx, st, dfs_num, dfs_low, onstack, all_sccs)
 return all_sccs
 O(V+E)

Kruskal MST
 subalg findroot (prev, v)
 while prev[v] != v: prev[v] = prev[prev[v]]
 return v
 subalg findedge (prev, v1, v2)
 root1 = findroot (prev, v1)
 root2 = findroot (prev, v2)
 if root1 == root2: return false
 prev[root1] = root2
 return true
 Kruskal (G)
 list mst t-cost = 0
 prev = dict
 for v in vertices: prev[v] = v
 list edges: set added
 for (x,y), cost in costs(x,y)
 edge = tuple(x,y)
 if edge not in added:
 added.add(edge)
 edges.append(cost, x,y)
 edges.sort()
 for cost, x,y in edges:
 if findroot (prev, x) != findroot (prev, y):
 t-cost += cost
 return mst, t-cost
 O(E log V)

Prim MST (G)
 Priority queue pq
 dict edge-cost, parent
 set mst
 start = vertices[0]
 for v in vertices: edge-cost[v] = inf
 pq.insert(start, 0)
 while not pq.empty():
 u = pq.extract_min()
 mst.add(u)
 for v in Nout(u)
 if v not in mst and
 cost(u,v) < edge-cost[v]:
 parent[v] = u
 edge-cost[v] = cost(u,v)
 pq.insert(v, edge-cost[v])
 return parent
 O(E log V)

Belman-Ford -> shortest p. n-all
 (G, s) for 4 cost cycle + < cost(s)
 dictionary dist, prev
 for each vertex u in G: vertices
 dist[u] = infinity
 dist[s] = 0
 for i from 1 to n: vertices - 1
 for each edge (u,v) in edges
 if dist[u] + cost(u,v) < dist[v]
 dist[v] = dist[u] + cost(u,v)
 prev[v] = u
 return dist, prev
 O(V * E)

Min cost walk DP
 (G, s, max_edges)
 matrix dp: dp[max-edges][i][vertex]
 dp[0][s] = 0
 for k from 1 to max_edges do
 for v in G.vertices
 dp[k][v] = dp[k-1][v]
 for u in Nin(v)
 if dp[k-1][u] + cost(u,v) < dp[k][v]
 dp[k][v] = dp[k-1][u] + cost(u,v)
 return dp
 O(K * V * E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> max len path
 longest time for activity
 (G) list order = top-sort(G)
 dict dist, prev
 for v in G.vertices
 dist[v] = -infinity
 dist[start] = 0
 for u in order
 if dist[u] != inf
 for v in Nout(u) do
 if dist[u] + cost(u,v) > dist[v]
 dist[v] = dist[u] + cost(u,v)
 prev[v] = u
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

DFS -> min len path (G, s)
 queue q: dist, prev, visited
 g: enqueue(s)
 while not q.empty():
 x = g.dequeue()
 for y in Nout(x) do:
 if y not in visited:
 visited.add(y)
 dist[y] = dist[x] + 1
 prev[y] = x
 return dist, prev
 O(V+E)

Max Flow of min cost

Find max flow while priorit cheap paths
 G, s, t, capacity, cost
 ResidualGraphs GR
 total-flow = 0 total-cost = 0
 // Sh. path based on cost Bell-ford
 while path = BF(GR, s, t) exists
 int bottleneck = min-cap along path
 for each edge (u, v) in path do
 GR.capacity[u][v] -= bottleneck
 GR.capacity[v][u] += bottleneck
 total-cost += cost(u, v) * bottleneck
 return total-flow, total-cost
 O(n^2 * (V * E))

Flow Problem



data flow network for all paths
 data flow network for all paths
 step naive card max path

Shortest cycle of < 0 total cost

matrix dp[k][u][v] = 0 prev[u+1][v] = none
 dp[0][s][s] = 0
 for k = 1, n vertices // min cost walks of len = k
 for each vertex u in vertices
 for each v in vertices
 if dp[k-1][u][v] != inf
 if dp[k-1][u][v] + cost(u, v) < dp[k][u][v]
 dp[k][u][v] = dp[k-1][u][v] + cost(u, v)
 if dp[k][t][t] < 0
 return dp[k][t][t]
 // find shortest cycle with < 0 t.c. (k1 s t k2 t s)
 min-total-len = 0
 for k1 from 1 to n vertices
 for k2 from 1 to n vertices
 if dp[k1][t][s] != inf and dp[k1+k2][s][s] != inf
 cost = dp[k1][t][s] + cost(s, t) + dp[k2][s][s]
 total-cycle-cost = cost + cost(s, t)
 if cost < 0
 min-total-len = k1 + k2 // update min-total-len
 if min-total-len != inf return it else "no cycle"

Forward Pass

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict ES, EF
 for v in order
 ES[v] = 0
 for p in NIn(v)
 ES[v] = max(ES[p], EF[p])
 EF[v] = ES[v] + duration(v)
 return ES, EF
 stack (ES, LF)
 dict slack
 for v in vertices
 slack[v] = ES[v] - EF[v]
 return slack
 O(V * E)

Backward Pass

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict LS, LF
 // total proj deadline (max EF)
 int pr-deadl = 0
 for v in DAG-vertices
 pr-deadl = max(pr-deadl, EF[v])
 for v in rev-order
 // if no NOut(v) latest finish = pr-deadl
 if NOut(v) empty LF[v] = pr-deadl
 else // LF is min of LS
 LF[v] = inf
 for s in NOut(v)
 LF[v] = min(LF[v], LS[s])
 LS[v] = LF[v] - duration
 return LS, LF
 O(V * E)

Path from matrix

(matrix, G, s, t, k, min-steps)
 path: list cur = t k = 0 steps
 while k > 0 do
 if cur == source and dp[k][cur][cur] == 0
 break
 local found-pre = false
 for each u in NIn(cur)
 if dp[k-1][u][cur] != inf
 if dp[k-1][u][cur] == dp[k-1][u][cur] + cost(u, cur)
 path.prepend(u)
 found = true
 return path (if found = true)
 O(E * S)

Path from matrix

for k = 1, n vertices + 1
 if dp[k][s][s] < 0 return k
 return "No cycle s to s"

Flow Problem

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict ES, EF
 for v in order
 ES[v] = 0
 for p in NIn(v)
 ES[v] = max(ES[p], EF[p])
 EF[v] = ES[v] + duration(v)
 return ES, EF
 stack (ES, LF)
 dict slack
 for v in vertices
 slack[v] = ES[v] - EF[v]
 return slack
 O(V * E)

Backward Pass

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict LS, LF
 // total proj deadline (max EF)
 int pr-deadl = 0
 for v in DAG-vertices
 pr-deadl = max(pr-deadl, EF[v])
 for v in rev-order
 // if no NOut(v) latest finish = pr-deadl
 if NOut(v) empty LF[v] = pr-deadl
 else // LF is min of LS
 LF[v] = inf
 for s in NOut(v)
 LF[v] = min(LF[v], LS[s])
 LS[v] = LF[v] - duration
 return LS, LF
 O(V * E)

Path from matrix

(matrix, G, s, t, k, min-steps)
 path: list cur = t k = 0 steps
 while k > 0 do
 if cur == source and dp[k][cur][cur] == 0
 break
 local found-pre = false
 for each u in NIn(cur)
 if dp[k-1][u][cur] != inf
 if dp[k-1][u][cur] == dp[k-1][u][cur] + cost(u, cur)
 path.prepend(u)
 found = true
 return path (if found = true)
 O(E * S)

Path from matrix

for k = 1, n vertices + 1
 if dp[k][s][s] < 0 return k
 return "No cycle s to s"

Flow Problem

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict ES, EF
 for v in order
 ES[v] = 0
 for p in NIn(v)
 ES[v] = max(ES[p], EF[p])
 EF[v] = ES[v] + duration(v)
 return ES, EF
 stack (ES, LF)
 dict slack
 for v in vertices
 slack[v] = ES[v] - EF[v]
 return slack
 O(V * E)

Backward Pass

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict LS, LF
 // total proj deadline (max EF)
 int pr-deadl = 0
 for v in DAG-vertices
 pr-deadl = max(pr-deadl, EF[v])
 for v in rev-order
 // if no NOut(v) latest finish = pr-deadl
 if NOut(v) empty LF[v] = pr-deadl
 else // LF is min of LS
 LF[v] = inf
 for s in NOut(v)
 LF[v] = min(LF[v], LS[s])
 LS[v] = LF[v] - duration
 return LS, LF
 O(V * E)

Path from matrix

(matrix, G, s, t, k, min-steps)
 path: list cur = t k = 0 steps
 while k > 0 do
 if cur == source and dp[k][cur][cur] == 0
 break
 local found-pre = false
 for each u in NIn(cur)
 if dp[k-1][u][cur] != inf
 if dp[k-1][u][cur] == dp[k-1][u][cur] + cost(u, cur)
 path.prepend(u)
 found = true
 return path (if found = true)
 O(E * S)

Path from matrix

for k = 1, n vertices + 1
 if dp[k][s][s] < 0 return k
 return "No cycle s to s"

Flow Problem

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict ES, EF
 for v in order
 ES[v] = 0
 for p in NIn(v)
 ES[v] = max(ES[p], EF[p])
 EF[v] = ES[v] + duration(v)
 return ES, EF
 stack (ES, LF)
 dict slack
 for v in vertices
 slack[v] = ES[v] - EF[v]
 return slack
 O(V * E)

Backward Pass

obs earliest start time each proj can begin
 list order = Topo-Sort (DAG)
 dict LS, LF
 // total proj deadline (max EF)
 int pr-deadl = 0
 for v in DAG-vertices
 pr-deadl = max(pr-deadl, EF[v])
 for v in rev-order
 // if no NOut(v) latest finish = pr-deadl
 if NOut(v) empty LF[v] = pr-deadl
 else // LF is min of LS
 LF[v] = inf
 for s in NOut(v)
 LF[v] = min(LF[v], LS[s])
 LS[v] = LF[v] - duration
 return LS, LF
 O(V * E)

Path from matrix

(matrix, G, s, t, k, min-steps)
 path: list cur = t k = 0 steps
 while k > 0 do
 if cur == source and dp[k][cur][cur] == 0
 break
 local found-pre = false
 for each u in NIn(cur)
 if dp[k-1][u][cur] != inf
 if dp[k-1][u][cur] == dp[k-1][u][cur] + cost(u, cur)
 path.prepend(u)
 found = true
 return path (if found = true)
 O(E * S)

Path from matrix

for k = 1, n vertices + 1
 if dp[k][s][s] < 0 return k
 return "No cycle s to s"